

A Course End Project Report on

Database Design and Management for Hotel Operations

A9512 – Database Management Systems Laboratory

Submitted by

P. Koushik	25881A0516
K. Mounvika Chowdary	25881A0523
S. Nikhil	25881A0528

Course Facilitator

Ms. Shaik Shahanaz
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Autonomous Institute, Affiliated to **JNTUH**,
Approved by **AICTE**, Accredited by **NAAC with A++ Grade**,
Kacharam, Shamshabad, Hyderabad – 501 218,
Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Database Design and Management for Hotel Operations"** is carried out by **"P. Koushik"**, **"K. Mounvika"** and **"S. Nikhil"** with Roll Numbers **"25881A0516"**, **"25881A0523"** and **"25881A0528"** towards **"A9512 – Database Management Systems Laboratory"** course in partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering during the Academic year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Shaik Shahanaz Assistant Professor, CSE	Dr. R. Karthikeyan Professor, CSE	Prof. Y. Vijayalata Department of CSE

Contents

Abstract 1
1 Introduction 2
2 Problem Analysis 2
2.1 Problem Statement 2
2.2 Users and Assumptions 2
2.3 Functional Requirements 2
3 ER Model 3
3.1 Entities and Attributes 3
3.2 Relationships 3
3.3 Cardinality and Participation 3
3.4 ER Diagram 3
4 Relational Model 5
4.1 Relational Schema 5
4.2 Primary Keys and Foreign Keys 5
5 Normalization 5
5.1 Functional Dependencies 5
5.2 First Normal Form (1NF) 5
5.3 Second Normal Form (2NF) 6
5.4 Third Normal Form (3NF) 6
6 SQL DDL Commands 6
6.1 Table Creation 6
6.2 Constraints 7
6.3 ALTER Operation 7
6.4 DROP Operation 7
7 SQL DML Commands 7
7.1 INSERT 7
7.2 UPDATE 7
7.3 DELETE 8
8 SQL Queries 8
8.1 Basic SELECT Queries 8
8.2 Joins 8
8.3 Aggregate Functions 8
8.4 GROUP BY and HAVING 8
8.5 Subqueries 8
8.6 Views 9
9 Transactions and Concurrency 9
9.1 Transaction Management 9
9.2 COMMIT, ROLLBACK and SAVEPOINT 9
9.3 Concurrency Control 9
10 PL/SQL Implementation 9
10.1 Functions 9
10.2 Stored Procedures 10
10.3 Cursors 10
10.4 Triggers 10

11 File Organization and Indexing 10
11.1 File Organization 10
11.2 ISAM 10
11.3 B+ Tree Indexing 11
12 Testing 11
12.1 Test Cases 11
12.2 Normal and Boundary Cases 11
12.3 Invalid Input Testing 11
13 Reports 11
13.1 Reservation Report 11
13.2 Payment Report 11
13.3 Room Occupancy Report 11
13.4 Service Report 11
14 Results and Discussion 12
15 Conclusion 12
16 SDG Mapping 12
17 Future Scope 12
18 Bibliography 13

Abstract

The Hotel Management System is a database-oriented application designed to manage guests, rooms, reservations, staff, payments, and additional hotel services in an organized manner. The system stores related information in separate relations and connects them using primary and foreign keys. This reduces redundancy and makes hotel operations easier to maintain.

The project demonstrates important Database Management System concepts including ER modelling, relational schema design, normalization, SQL DDL and DML commands, joins, aggregate queries, subqueries, views, transactions, concurrency control, PL/SQL functions, procedures, cursors, triggers, file organization, indexing, and testing.

The system provides a structured database model for handling hotel reservations, room information, guest details, staff information, payments, and additional services.

Keywords: Hotel Management System, Database Management System, SQL, reservations, room management, payments, services, PL/SQL.

1 Introduction

A hotel management system requires the storage and processing of information about guests, rooms, room categories, reservations, staff, payments, and services. Maintaining these details manually can result in duplicate records, inconsistent information, and difficulty in retrieving required data.

The proposed Hotel Management System uses a relational database to organize hotel information. A guest can make multiple reservations, a room belongs to one room type, staff members handle reservations, and reservations can have a payment record. Guests can also use additional hotel services through service bookings.

The project applies Database Management System principles to design a consistent and efficient database suitable for common hotel operations.

2 Problem Analysis

2.1 Problem Statement

The objective is to design a database that manages hotel rooms, guests, reservations, staff, payments, and services while maintaining data consistency and reducing unnecessary duplication.

2.2 Users and Assumptions

The main users are hotel staff and administrators.

The system assumes that:

- Each guest has a unique Guest ID.
- Each room has a unique Room ID.
- Each reservation has a unique Reservation ID.
- Each staff member has a unique Staff ID.
- Each payment belongs to a reservation.
- Each service has a unique Service ID.

2.3 Functional Requirements

The system should:

- Store and retrieve guest details.
- Maintain room and room-type information.
- Create and manage reservations.
- Associate staff members with reservations.
- Record reservation payments.
- Maintain hotel services and service bookings.
- Generate reservation and payment reports.
- Display room occupancy information.
- Maintain data integrity using constraints.

3 ER Model

3.1 Entities and Attributes

The Hotel Management System contains the following eight entities:

- **Guest:** Guest ID, Guest Name, Email, Phone, Address.
- **Room Type:** Type ID, Type Name, Capacity, Price Per Night.
- **Room:** Room ID, Type ID, Room Number, Floor, Status.
- **Reservation:** Reservation ID, Guest ID, Room ID, Staff ID, CheckIn, CheckOut, Status.
- **Staff:** Staff ID, Staff Name, Role, Phone, Salary.
- **Payment:** Payment ID, Reservation ID, Amount, Payment Date, Payment Mode, Status.
- **Service Booking:** ServiceBooking ID, Guest ID, Service ID, Quantity, Service Date.
- **Service:** Service ID, Service Name, Service Cost.

3.2 Relationships

The main relationships in the system are:

- Guest makes Reservation.
- Room Type has Room.
- Room is assigned to Reservation.
- Staff handles Reservation.
- Reservation has Payment.
- Guest uses Service Booking.
- Service Booking is for Service.

3.3 Cardinality and Participation

The cardinalities represented in the ER model are:

- Guest to Reservation: 1:N.
- Room Type to Room: 1:N.
- Room to Reservation: 1:N.
- Staff to Reservation: 1:N.
- Reservation to Payment: 1:1.
- Guest to Service Booking: 1:N.
- Service Booking to Service: N:1.

3.4 ER Diagram

The ER diagram represents the entities, attributes, primary keys, relationships, and cardinalities of the Hotel Management System. It provides a graphical representation of the database structure and serves as the basis for converting the ER model into relational tables.

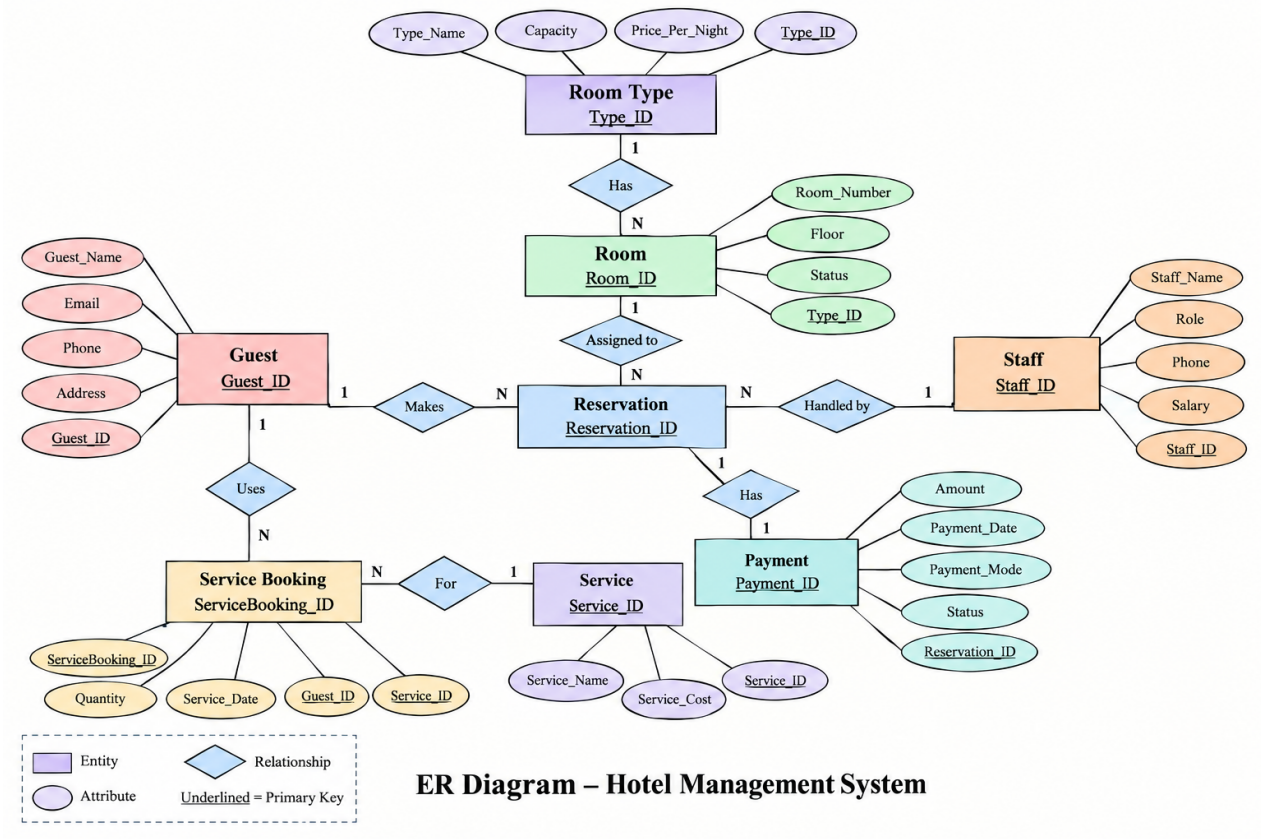


Figure 1: ER Diagram of Hotel Management System

4 Relational Model

4.1 Relational Schema

- GUEST (Guest ID, Guest Name, Email, Phone, Address)
- ROOM TYPE (Type ID, Type Name, Capacity, Price Per Night)
- ROOM (Room ID, Type ID, Room Number, Floor, Status)
- STAFF (Staff ID, Staff Name, Role, Phone, Salary)
- RESERVATION (Reservation ID, Guest ID, Room ID, Staff ID, CheckIn, CheckOut, Status)
- PAYMENT (Payment ID, Reservation ID, Amount, Payment Date, Payment Mode, Status)
- SERVICE (Service ID, Service Name, Service Cost)
- SERVICE BOOKING (ServiceBooking ID, Guest ID, Service ID, Quantity, Service Date)

4.2 Primary Keys and Foreign Keys

The primary keys are:

- Guest ID
- Type ID
- Room ID
- Staff ID
- Reservation ID
- Payment ID
- Service ID
- ServiceBooking ID

The foreign keys are:

- Type ID in ROOM.
- Guest ID, Room ID and Staff ID in RESERVATION.
- Reservation ID in PAYMENT.
- Guest ID and Service ID in SERVICE BOOKING.

5 Normalization

5.1 Functional Dependencies

The important functional dependencies are:

$$\begin{aligned}
\text{Guest ID} &\rightarrow \text{Guest Name, Email, Phone, Address} \\
\text{Room ID} &\rightarrow \text{Type ID, Room Number, Floor, Status} \\
\text{Reservation ID} &\rightarrow \text{Guest ID, Room ID, Staff ID, CheckIn, CheckOut, Status} \\
\text{Payment ID} &\rightarrow \text{Reservation ID, Amount, Payment Date, Payment Mode, Status}
\end{aligned}$$

5.2 First Normal Form (1NF)

All relations contain atomic values. Each column contains a single value and there are no repeating groups.

5.3 Second Normal Form (2NF)

The relations use appropriate primary keys and non-key attributes depend on the complete primary key. Separate tables are used for guests, rooms, staff, services, and payments.

5.4 Third Normal Form (3NF)

Non-key attributes do not depend on other non-key attributes. Room type information is stored separately from Room and service information is stored separately from Service Booking. This reduces insertion, deletion, and update anomalies.

6 SQL DDL Commands

6.1 Table Creation

```
CREATE TABLE Guest (
    Guest_ID NUMBER PRIMARY KEY,
    Guest_Name VARCHAR2(50) NOT NULL,
    Email VARCHAR2(80),
    Phone VARCHAR2(15),
    Address VARCHAR2(120)
);

CREATE TABLE Room_Type (
    Type_ID NUMBER PRIMARY KEY,
    Type_Name VARCHAR2(30) NOT NULL,
    Capacity NUMBER NOT NULL,
    Price_Per_Night NUMBER(10,2) NOT NULL
);

CREATE TABLE Room (
    Room_ID NUMBER PRIMARY KEY,
    Type_ID NUMBER NOT NULL,
    Room_Number VARCHAR2(10) UNIQUE NOT NULL,
    Floor NUMBER,
    Status VARCHAR2(20),
    FOREIGN KEY (Type_ID) REFERENCES Room_Type(Type_ID)
);

CREATE TABLE Staff (
    Staff_ID NUMBER PRIMARY KEY,
    Staff_Name VARCHAR2(50) NOT NULL,
    Role VARCHAR2(30),
    Phone VARCHAR2(15),
    Salary NUMBER(10,2)
);

CREATE TABLE Reservation (
    Reservation_ID NUMBER PRIMARY KEY,
    Guest_ID NUMBER NOT NULL,
    Room_ID NUMBER NOT NULL,
    Staff_ID NUMBER NOT NULL,
    CheckIn DATE NOT NULL,
    CheckOut DATE NOT NULL,
    Status VARCHAR2(20),
    FOREIGN KEY (Guest_ID) REFERENCES Guest(Guest_ID),
    FOREIGN KEY (Room_ID) REFERENCES Room(Room_ID),
    FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID)
);

CREATE TABLE Payment (
    Payment_ID NUMBER PRIMARY KEY,
    Reservation_ID NUMBER UNIQUE,
    Amount NUMBER(10,2) NOT NULL,
```

```
    Payment_Date DATE,
    Payment_Mode VARCHAR2(20),
    Status VARCHAR2(20),
    FOREIGN KEY (Reservation_ID)
        REFERENCES Reservation(Reservation_ID)
);

CREATE TABLE Service (
    Service_ID NUMBER PRIMARY KEY,
    Service_Name VARCHAR2(50) NOT NULL,
    Service_Cost NUMBER(10,2) NOT NULL
);

CREATE TABLE Service_Booking (
    ServiceBooking_ID NUMBER PRIMARY KEY,
    Guest_ID NUMBER NOT NULL,
    Service_ID NUMBER NOT NULL,
    Quantity NUMBER NOT NULL,
    Service_Date DATE,
    FOREIGN KEY (Guest_ID) REFERENCES Guest(Guest_ID),
    FOREIGN KEY (Service_ID) REFERENCES Service(Service_ID)
);
```

6.2 Constraints

Primary keys uniquely identify records. Foreign keys maintain referential integrity between related tables. NOT NULL and UNIQUE constraints prevent invalid or duplicate data.

6.3 ALTER Operation

```
ALTER TABLE Guest
ADD Gender VARCHAR2(10);
```

6.4 DROP Operation

```
DROP TABLE Service_Booking;
```

The DROP operation permanently removes the table and should therefore be used carefully.

7 SQL DML Commands

7.1 INSERT

```
INSERT INTO Guest
VALUES
(101, 'Rahul', 'rahul@gmail.com',
 '9876543210', 'Hyderabad');

INSERT INTO Room_Type
VALUES
(1, 'Deluxe', 2, 3500);

INSERT INTO Staff
VALUES
(501, 'Anita', 'Manager',
 '9876500000', 45000);
```

7.2 UPDATE

```
UPDATE Reservation
SET Status = 'Confirmed'
WHERE Reservation_ID = 1001;
```

7.3 DELETE

```
DELETE FROM Service_Booking
WHERE ServiceBooking_ID = 301;
```

8 SQL Queries

8.1 Basic SELECT Queries

```
SELECT * FROM Guest;

SELECT Room_Number, Status
FROM Room
WHERE Status = 'Available';

SELECT Guest_Name, Phone
FROM Guest
ORDER BY Guest_Name;
```

8.2 Joins

```
SELECT g.Guest_Name,
       r.Reservation_ID,
       r.CheckIn,
       r.CheckOut
FROM Guest g
JOIN Reservation r
ON g.Guest_ID = r.Guest_ID;

SELECT r.Room_Number,
       rt.Type_Name,
       rt.Price_Per_Night
FROM Room r
JOIN Room_Type rt
ON r.Type_ID = rt.Type_ID;
```

8.3 Aggregate Functions

```
SELECT COUNT(*) AS Total_Guests
FROM Guest;

SELECT SUM(Amount) AS Total_Payment
FROM Payment;

SELECT AVG(Price_Per_Night) AS Average_Room_Price
FROM Room_Type;
```

8.4 GROUP BY and HAVING

```
SELECT Status, COUNT(*) AS Total
FROM Reservation
GROUP BY Status
HAVING COUNT(*) > 0;
```

8.5 Subqueries

```
SELECT Room_Number,
       Price_Per_Night
FROM Room r
JOIN Room_Type rt
ON r.Type_ID = rt.Type_ID
WHERE Price_Per_Night >
(
    SELECT AVG(Price_Per_Night)
```

```
    FROM Room_Type
);
```

8.6 Views

```
CREATE OR REPLACE VIEW Reservation_Details AS
SELECT g.Guest_Name,
       r.Reservation_ID,
       r.CheckIn,
       r.CheckOut,
       r.Status
FROM Guest g
JOIN Reservation r
ON g.Guest_ID = r.Guest_ID;

SELECT * FROM Reservation_Details;
```

9 Transactions and Concurrency

9.1 Transaction Management

A transaction is a logical unit of database operations. A reservation and its payment can be treated as related operations so that the database remains consistent.

9.2 COMMIT, ROLLBACK and SAVEPOINT

```
UPDATE Reservation
SET Status = 'Confirmed'
WHERE Reservation_ID = 1001;

SAVEPOINT reservation_update;

UPDATE Reservation
SET Status = 'Cancelled'
WHERE Reservation_ID = 1001;

ROLLBACK TO reservation_update;

COMMIT;
```

COMMIT permanently saves the transaction. ROLLBACK cancels uncommitted changes. SAVEPOINT creates a point to which a transaction can be partially rolled back.

9.3 Concurrency Control

When several users access the database at the same time, concurrency control prevents inconsistent updates. Database transactions and locking mechanisms help maintain isolation and consistency.

10 PL/SQL Implementation

10.1 Functions

```
CREATE OR REPLACE FUNCTION get_room_price(
    p_type_id NUMBER
)
RETURN NUMBER
IS
    v_price NUMBER;
BEGIN
    SELECT Price_Per_Night
    INTO v_price
    FROM Room_Type
```

```
    WHERE Type_ID = p_type_id;

    RETURN v_price;
END;
/
```

10.2 Stored Procedures

```
CREATE OR REPLACE PROCEDURE update_reservation_status(
    p_reservation_id NUMBER,
    p_status VARCHAR2
)
IS
BEGIN
    UPDATE Reservation
    SET Status = p_status
    WHERE Reservation_ID = p_reservation_id;

    COMMIT;
END;
/
```

10.3 Cursors

```
DECLARE
   CURSOR guest_cursor IS
       SELECT Guest_ID, Guest_Name
       FROM Guest;
BEGIN
   FOR g IN guest_cursor LOOP
      DBMS_OUTPUT.PUT_LINE(
          g.Guest_ID || ' ' || g.Guest_Name
      );
   END LOOP;
END;
/
```

10.4 Triggers

```
CREATE OR REPLACE TRIGGER check_reservation_dates
BEFORE INSERT OR UPDATE ON Reservation
FOR EACH ROW
BEGIN
   IF :NEW.CheckOut <= :NEW.CheckIn THEN
      RAISE_APPLICATION_ERROR(
          -20001,
          'CheckOut must be after CheckIn'
      );
   END IF;
END;
/
```

11 File Organization and Indexing

11.1 File Organization

File organization determines how records are stored physically. Sequential, heap, and hashed organizations can be used depending on the access pattern.

11.2 ISAM

Indexed Sequential Access Method maintains records in sorted order with index structures. It supports both sequential processing and direct access through indexes.

11.3 B+ Tree Indexing

B+ trees are balanced index structures widely used for database searching. Leaf nodes contain indexed key values and pointers to records or data blocks. They provide efficient search and range queries.

Example:

```
CREATE INDEX idx_guest_name
ON Guest(Guest_Name);
```

12 Testing

12.1 Test Cases

Test	Input / Operation	Expected Result
T1	Insert valid guest	Guest record is inserted
T2	Duplicate Guest ID	Primary-key error
T3	Insert valid reservation	Reservation is created
T4	Invalid Guest ID in reservation	Foreign-key error
T5	CheckOut before CheckIn	Trigger rejects operation
T6	Update reservation status	Status is updated
T7	Delete service booking	Matching booking is removed

12.2 Normal and Boundary Cases

Normal cases include valid guest, room, reservation, and payment values. Boundary cases include minimum quantities, valid date limits, and maximum expected field lengths.

12.3 Invalid Input Testing

Invalid foreign keys, duplicate primary keys, missing mandatory values, and incorrect reservation dates should be rejected by constraints or triggers.

13 Reports

13.1 Reservation Report

The reservation report displays guest names, reservation IDs, rooms, check-in dates, check-out dates, and reservation status.

13.2 Payment Report

The payment report displays payment ID, reservation ID, amount, payment date, payment mode, and payment status.

13.3 Room Occupancy Report

The room occupancy report can be generated from room and reservation information to identify occupied and available rooms.

13.4 Service Report

The service report displays service bookings along with the guest, service name, quantity, and service date.

14 Results and Discussion

The database design organizes hotel information into related relations and applies primary-key and foreign-key constraints for data integrity.

SQL queries provide mechanisms for retrieval, filtering, joining, aggregation, and reporting. PL/SQL features demonstrate procedural processing within the database.

The ER model and normalized relational model reduce redundancy and make the database easier to maintain. Transactions provide controlled updates, while indexing supports efficient retrieval.

15 Conclusion

The Hotel Management System demonstrates how Database Management System concepts can be applied to a practical hotel environment.

The system models guests, rooms, reservations, staff, payments, and services using an organized relational structure.

The project covers database design, normalization, SQL, PL/SQL, transactions, indexing, and testing. The resulting design provides a structured foundation that can be extended into a complete hotel application.

16 SDG Mapping

The project supports the following Sustainable Development Goals through efficient digital information management:

- **SDG 9 – Industry, Innovation and Infrastructure:** encourages digital and database-based management systems.
- **SDG 12 – Responsible Consumption and Production:** organized information can support efficient use of rooms and hotel resources.
- **SDG 8 – Decent Work and Economic Growth:** structured hotel operations can support efficient service management and business processes.

17 Future Scope

The system can be extended with:

- Online hotel booking.
- Customer login and authentication.
- Automatic room availability checking.
- Online payment integration.
- Email and SMS notifications.
- Dynamic billing.
- Housekeeping management.
- Inventory management.
- Analytics dashboards.
- Web and mobile applications.

A web or mobile application can be developed on top of the database so that guests and hotel staff can access appropriate functions through secure interfaces.

18 Bibliography

1. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, *Database System Concepts*.
2. Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*.
3. Oracle Database Documentation, SQL and PL/SQL reference material.
4. Course material for Database Management Systems, Department of Computer Science and Engineering, Vardhaman College of Engineering.